

CPCS 202 - Programming I

01 - Introduction to Programming and Java

Objectives

- Software Development Process
- Java and JDK
- Algorithms and Flowchart
- IDEs

Types of Software

Three basic types of Software

- 1. System software** - provides core functions such as operating systems, disk management, utilities, etc.
- 2. Programming software** - programmer's tools such as text editors, compilers, linkers, debuggers and other tools to create code.
- 3. Application software (applications or apps)** - to help users perform tasks. Word processors, media players, mobile applications

Steps in Software Development

1. Requirements Analysis (in Natural Language)
2. Design
3. Implementation
4. Testing
5. Deployment (Installation)
6. Maintenance

Software Design

- Software design is a process to transform requirements (given in Natural Language) into some easy-to-understand form, which helps the programmer in implementation
- **Algorithms** are specified or selected or created during design

Algorithm

- A step-by-step procedure used for solving a problem or performing a computation
- Examples of algorithms
 - Searching and sorting algorithms
 - Encryption Algorithms
 - Machine Learning Algorithms
- Algorithms are described in **Pseudocode** or using **Flowcharts**

Pseudocode

- An informal way of describing the steps in an algorithm
- Does not have any strict programming language syntax
- Used to create an outline or a rough draft of an algorithm or program

Flowchart

- A flowchart is a visual description of the steps involved in an algorithm
- They are easy to create and are easy to understand

Java Programming Language

- High-level, object-oriented programming language
- First released by Sun Microsystems in 1995
- Enables programmers to write once, run anywhere without the need to recompile
- Java is **portable** (programs written for one platform must run similarly on any other platform)
- Syntax of Java is similar to C and C++

Java Virtual Machine (JVM) and Bytecode

- Java code is compiled to an intermediate representation called Java **bytecode**, instead of directly to architecture-specific machine code
- Java bytecode instructions are executed on a Java virtual machine (**JVM**)
- This **bytecode** that can run on any **JVM** regardless of the underlying computer architecture
- End-users commonly use a Java Runtime Environment (**JRE**) installed on their device to run their standalone Java applications

Java Development Kit (JDK)

- Java Development Kit (JDK) is used for developing Java applications
- It provides the following software
 - Virtual machine (JVM)
 - Compiler (**javac**)
 - Performance monitoring tools
 - Debugger
 - Java Runtime Environment (JRE)
 - Other utilities useful for Java programmers

Software Development Process in Java

- Java development starts with creating a file (**.java*) which contains the Java source code
- This is then compiled into Java bytecode using *javac*, the Java Compiler which converts the **.java* file to a **.class* file
- The **JRE** is then used to execute the **.class* file and see the output

Integrated Development Environments (IDEs)

- IDEs are tools used to simplify software development
- IDEs provide a simple user interface that has everything a developer needs to write code
- IDEs provide the following
 - code **editor** to write and edit code
 - **debugger** to debug code
 - code **compiler** to compile code
 - **run environment** to execute the compiled code
 - automate some software development tasks.

Popular IDEs for Java Development

Some popular IDEs for Java Development

1. [Netbeans](#)
2. [Eclipse](#)
3. [IntelliJ IDEA](#)

Note : Follow the links to download these IDEs and to obtain more information on these IDEs

A list of the best IDEs for 2022 for Java Development can be found [here](#)

Structure of a Java program

```
import java.util.Scanner;
public class MyFirstProgram {
    /*
     * This is my first Java Program
     */
    public static void main(String[] args) {
        // The line below displays the message "Welcome to Java!"
        // on the screen
        System.out.println("Welcome to Java!");
    }
}
```

Structure of a Java program – Class name

```
import java.util.Scanner;
public class MyFirstProgram {
    /*
     * This is my first Java Program
     */
    public static void main(String[] args) {
        // The line below displays the message "Welcome to Java!"
        // on the screen
        System.out.println("Welcome to Java!");
    }
}
```

- Every Java Program must have at least one class
- Note the naming convention used to name the class

Structure of a Java program – main method

```
import java.util.Scanner;
public class MyFirstProgram {
    /*
     * This is my first Java Program
     */
    public static void main(String[] args) {
        // The line below displays the message "Welcome to Java!"
        // on the screen
        System.out.println("Welcome to Java!");
    }
}
```

- Every Java Program must has exactly one main method
- This is where the execution of any Java program first starts

Structure of a Java program - Statement

```
import java.util.Scanner;
public class MyFirstProgram {
    /*
     * This is my first Java Program
     */
    public static void main(String[] args) {
        // The line below displays the message "Welcome to Java!"
        // on the screen
        System.out.println("Welcome to Java!");
    }
}
```

- Executable instructions that tell a compiler what action to perform
- All Java statements are terminated with a semicolon (;)

Structure of a Java program - Comments

```
import java.util.Scanner;
public class MyFirstProgram {
    /*
    * This is my first Java Program
    */
    public static void main(String[] args) {
        // The line below displays the message "Welcome to Java!"
        // on the screen
        System.out.println("Welcome to Java!");
    }
}
```

- Used by programmers to document their code for easier understanding
- Two types of comments – Multi-line comments and single line comments
- Comments are ignored by the compiler

Structure of a Java program - Blocks

```
import java.util.Scanner;  
public class MyFirstProgram {  
    /*  
    * This is my first Java Program  
    */  
    public static void main(String[] args) {  
        // The line below displays the message "Welcome to Java!"  
        // on the screen  
        System.out.println("Welcome to Java!");  
    }  
}
```

- Set of code enclosed within two curly braces - { and }
- Used to group one or more Java statements

Structure of a Java program – import statements

```
import java.util.Scanner;
public class MyFirstProgram {
    /*
     * This is my first Java Program
     */
    public static void main(String[] args) {
        // The line below displays the message "Welcome to Java!"
        // on the screen
        System.out.println("Welcome to Java!");
    }
}
```

- Used to import (or include) external code like classes or packages into your code

Types of Programming Errors

1. Syntax Errors (or compile time errors)
2. Runtime Errors
3. Logical Errors

Programming Errors – Syntax Errors

- Occur when code does not follow language syntax rules
- Detected during compilation (or by the IDE)
- Easy to spot and easy to rectify
- Compiler (or IDE) indicates where the error is

```
public class MyFirstProgram {  
    public static void Main(String[] args) {  
        System.out.println("Any Message to be printed")  
    }  
}
```

Incorrect case
of letter 'm'

Incorrect
quote

Missing
semicolon

Programming Errors – Runtime Errors

- Occur (or are detected) during the execution of the program
- May be discovered when user enters invalid data
- Examples
 - Runtime error caused by diving by zero
 - User enters a String when program is expecting an int

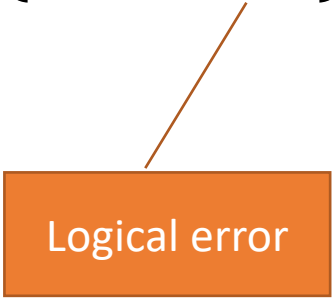
```
public class MyFirstProgram {  
    public static void main(String[] args) {  
        System.out.println((3+5)/(6-3-3));  
    }  
}
```

Divide by zero

Programming Errors – Logical Errors

- Hardest to detect and rectify
- A program with logical errors will compile and execute successfully but may not produce the expected result

```
public class MyFirstProgram {  
    public static void main(String[] args) {  
        System.out.println("52 + 128 = " + (52 + 138));  
    }  
}
```



Logical error